28

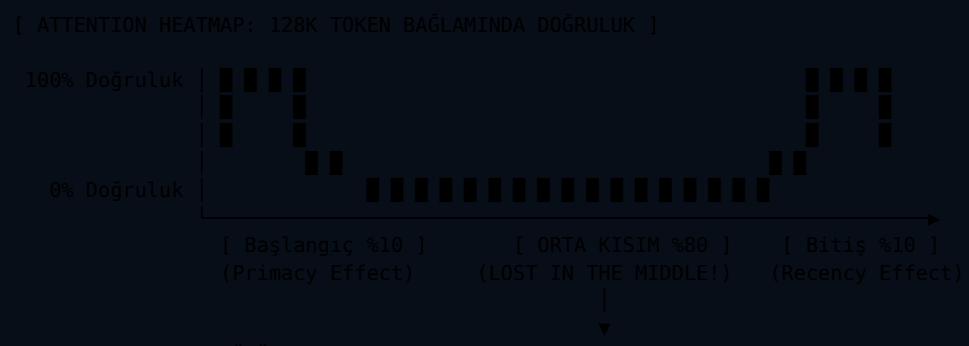
BÖLÜM: İLERI DÜZEY RAG (ADVANCED RAG) & BILGI GETIRME OPTIMIZASYONU | MODULE: ADVANCED RAG & INFORMATION RETRIEVAL OPTIMIZATION

Bağlam Sıkıştırma (Context Compression) & 'Lost in the Middle' Problemi

Context Compression & Mitigating the 'Lost in the Middle' Attention Deficit

🤖 AI Mimarisi & Veri Akış Şeması

🤖 AI Architecture & Flow



💡 Aptala Anlatım (ELI5 Metaforu)

💡 ELI5 Metaphor

🇹🇷 TÜRKÇE (ELI5)

Öğretmeniniz size 50 sayfalık bir kitap verip 'Yarın sınav var' dediğinde ne yaparsınız? Kitabın ilk sayfasını dikkatle okursunuz (Aklınızda kalır). Son sayfasını da yeni okuduğunuz için hatırlarsınız. Ama 25. sayfadaki o küçük detayı kesinlikle unutursunuz! Yapay zeka modelleri de tıpkı insanlar gibi dikkat dağınıklığı yaşar (Lost in the Middle): Bağlam uzadıkça ortada duran bilgileri görmezden gelir. Bağlam sıkıştırma ise kitaptaki tüm gereksiz 've, veya, çünkü' kelimelerini atıp sadece sınavda çıkacak 3 cümleyi modelin tam gözünün önüne koymaktır!

🇬🇧 ENGLISH (ELI5)

When crammed with a 200-page dossier before a trial, a human attorney vividly recalls the opening argument (Primacy Effect) and the final conclusion (Recency Effect), but glazes over fine print buried on page 97. Transformer attention mechanisms suffer from the identical cognitive bias: 'Lost in the Middle' (Liu et al., Stanford/Berkeley). Model recall plummets when critical facts reside in the middle of long contexts. Context Compression strips conversational fluff, while Context Reordering places the highest-ranked evidence at the extreme boundaries!

🔍 MAANG Seviyesi Teknik Derinlik

🇹🇷 MAANG DERİNLEMESİNE ANALİZ

Dikkat Mekanizması ve Sıkıştırma Teknikleri: 1) Lost in the Middle Fenomeni: U-şeklinde bir doğruluk eğrisidir. Pozisyonel embedding'ler (RoPE / ALiBi) ne kadar gelişirse gelişsin, softmax normalizasyonu uzun bağlamlarda orta katman dikkatini seyreltir. 2) Re-ordering (Sıralama Hilesi): En yüksek skorlu dokümanları ortaya değil; sırasıyla [1. Doküman en başa, 2. Doküman en sona, 3. Doküman ikinci sıraya] gelecek şekilde alternatifli dizin. 3) LLMLingua & Context Compression: Küçük bir encoder ile gereksiz doldurma kelimeleri (stop words) silinerek token sayısı %50 sıkıştırılır; hem maliyet yarıya iner hem de modelin dikkati odaklanır.

🔍 MAANG-Level Technical Deep-Dive

🇬🇧 MAANG TECHNICAL DEEP-DIVE

Transformer Attention Degradation & Mitigation: 1) The Lost-in-the-Middle Phenomenon: Rigorous empirical studies (Liu et al.) demonstrate that LLM recall forms a distinct U-shaped curve across long contexts. Softmax attention distributions dilute over long token sequences, heavily favoring prefix and suffix tokens. 2) Attention-Aware Re-ordering: Place the 1 most authoritative chunk at the very beginning of the context block, the 2 chunk at the very end, and weaker chunks in the center. 3) Context Compression (LLMLingua): Employs a compact token pruner to strip low-entropy tokens, condensing context volume by 30-50% without factual loss.

⚡ Java 25, Spring AI & LangChain4j Kodu

⚡ Java 25 & OpenAI Implementation

```java
// Java 25: Lost in the Middle Korumalı Bağlam Yeniden Sıralayıcı (Context Re-orderer)
import java.util.ArrayList;
import java.util.List;

public class ContextReorderingOptimizer {
    /**
     * En iyi dokümanları U-şeklindeki attention eğrisinin uçlarına yerleştirir.
     * Sıralama Mantığı: En iyi doküman başa (#1), ikinci en iyi doküman sona (#2)...
     */
    public static <T> List<T> reorderForAttention(List<T> sortedCandidates) {
        if (sortedCandidates.size() <= 2) return sortedCandidates;

        List<T> reordered = new ArrayList<>(sortedCandidates.size());
        int left = 0;
        int right = sortedCandidates.size() - 1;
        boolean toggle = true;

        while (left <= right) {
            if (toggle) {
                reordered.add(0, sortedCandidates.get(left++)); // Başa ekle
            } else {
                reordered.add(sortedCandidates.get(left++));     // Sona ekle
            }
            toggle = !toggle;
        }
        return reordered;
    }
}

void main() {
    List<String> topDocs = List.of("Doc1 (En İyi)", "Doc2 (İyi)", "Doc3 (Orta)", "Doc4 (Zayıf)");
    var optimized = ContextReorderingOptimizer.reorderForAttention(topDocs);

    IO.println("Attention Odaklı Bağlam Sıralaması:");
    optimized.forEach(IO::println);
}
```

⚠️ Halüsinasyon, Güvenlik & Mülakat Tuzağı

🇹🇷 MÜLAKAT TUZAĞI VE HALÜSİNASYON

Mülakat Hatası: '128k context window var, RAG'den gelen 30 dokümanın hepsini prompt'a ekledim' demek mülakatın sonu olur. Fazla bağlam hem maliyeti katlar, hem yanıt süresini 5 saniyeye çıkarır, hem de 'Lost in the Middle' yüzünden modelin doğru cevabı ıskalamasına yol açar. İdeal bağlam 3 ila 5 filtrelenmiş parçadır.

⚠️ Hallucination & Interview Trap

🇬🇧 INTERVIEW TRAP & HALLUCINATION RISK

Interview failure: Stuffing 30 retrieved chunks into prompt context under the naive assumption that 'the LLM will figure it out'. Excessive context inflates latency, drains budget, and guarantees retrieval failure due to attention dilution. Keep production context strictly capped to top 3-5 verified chunks.

🎯 Mülakatta Hatırlanacak Altın Çıkarımlar

- LLM'ler bağlamın başındaki ve sonundaki bilgileri çok iyi, ortasındakileri çok kötü hatırlar.
- En kritik kanıtlar bağlam blokunun en başına ve en sonuna yerleştirilmelidir.
- Bağlam sıkıştırma, token maliyetini %50 düşürürken modelin odaklanmasını sağlar.

🎯 Key Takeaways for the Interview

- Transformers exhibit a strong U-shaped attention bias, neglecting information buried in the middle.
- Structure retrieved context so that primary evidence brackets the start and finish of the prompt.
- Context compression slashes token bills while intensifying semantic attention on vital facts.